

MQTT Security

Frank Walsh

+ New Subscription

/#

QoS 0

Plaintext ▾

All

Topic: /niisten4/862078078713156/off/pub2 QoS: 0

□□□□□□

2024-02-07 15:46:47:729

Topic: /sahuifa QoS: 0

insert into tjddcs.t_node1_group1_TAG121212 using tjddcs.stjddcs tags (1) values (1707320807272, 0)

2024-02-07 15:46:47:729

Topic: /neuron/11/modbus-tcp-test/group2 QoS: 0

{"node": "modbus-tcp-test", "group": "group2", "timestamp": 1707320792603, "values": {}, "errors": {"tag7": 3002, "tag8": 3002, "tag9": 3002, "tag10": 3002, "tag5": 3002, "tag6": 3002}}

2024-02-07 15:46:47:730

Topic: /merakimv/Q2GV-EY5H-6TFX/light QoS: 0

{"lux": 646.6}

2024-02-07 15:46:47:749

Topic: /neuron/MQTT/机床/西门子PLC QoS: 0

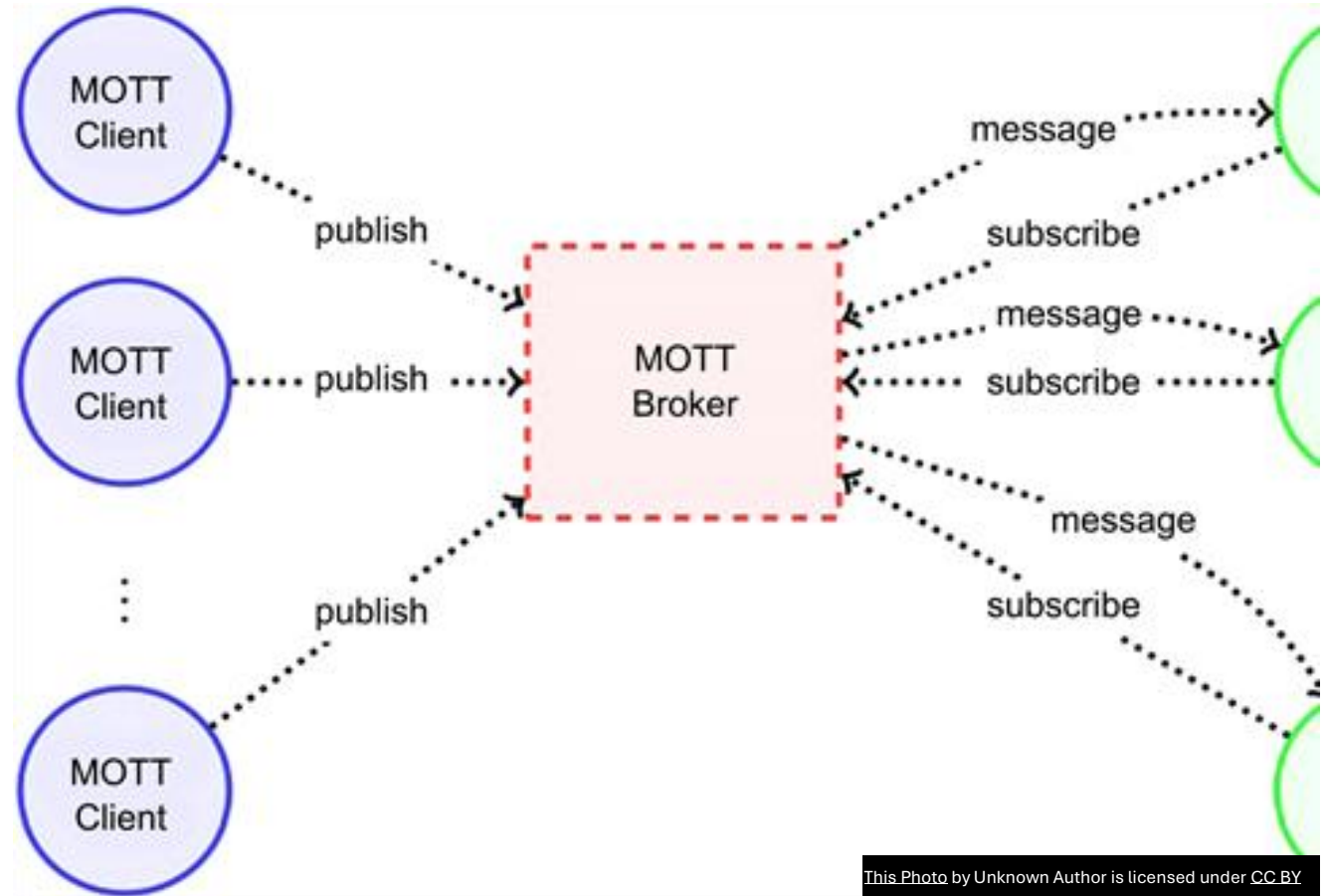


Security in IoT

- Internet of Things presents new security implementation challenges
 - IoT devices have limited computing power and memory capacity
 - Cryptographic algorithms may require more resources than constrained IoT devices possess
 - Critical security issues may require updates to be rolled out to all devices simultaneously can be affected by unreliable networks on which many IoT devices
- **Security is a critical concern for developers of IoT applications.**

Security in MQTT

- We can take a “layered” approach over the protocol stack
- MQTT specifies only a few security mechanisms
- MQTT uses other accepted security standards at the various levels:
 - E.g.SSL/TLS for transport security.
- **Network Layer:** Use a VPN for all communication between clients and brokers. Gateway-based applications, the gateway is connected to devices on the one hand and with the broker over VPN on the other side.
- **Transport Layer:** TLS/SSL
- **Application Layer:** Authorisation using username/password provided by the Protocol



This Photo by Unknown Author is licensed under CC BY

MQTT Authentication

- The MQTT protocol provides username and password fields in the CONNECT message for authentication
 - Username is an UTF-8 encoded string.
 - Password is binary data with a maximum of 65535 bytes.
- The MQTT broker evaluates credentials and returns one of the following return codes:
 - 0 – Connection Accepted
 - 4 – Connection Refused, bad username/password
 - 5 – Connection Refused, not authorised
- Username/password sent as plain text - **Secure transmission of usernames and passwords requires transport encryption**

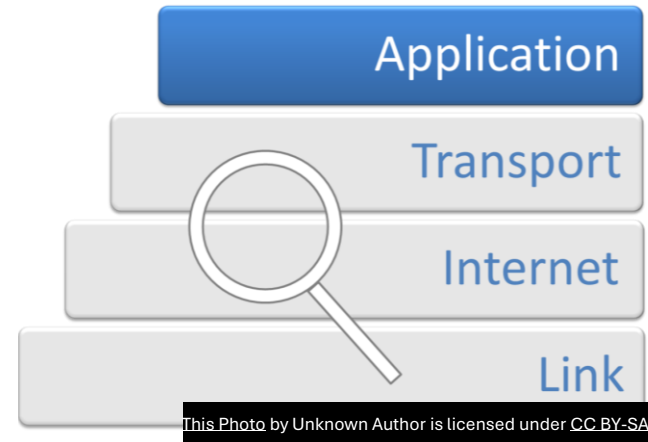
Transport Layer Security (TLS)

- TLS provide a secure communication channel between a client and a server **at Transport Layer**.
- TLS is a cryptographic protocol which use a handshake mechanism to negotiate various parameters to create a secure connection between the client and the server.
- **Servers provide a certificate** (typically issued by a trusted authority) that clients use to verify the identity of the server.
- While the additional CPU usage is typically negligible on the broker, **small constrained devices are not designed for computation-intensive tasks**.
- If TLS is not possible, **payload encryption and at least hash or encrypt the password in the CONNECT message of your client**.



Application Level Encryption

- End to End encryption between publisher and subscriber
- Encrypt the payload without having to configure the MQTT broker
 - Means publisher and subscriber applications have to “know” encryption/decryption method. Has to be coded
- Possible alternative for constrained devices that cannot support TLS
- Possible (less secure) alternative for using public brokers.



```
from cryptography.fernet import Fernet

def encrypt_payload(payload):
    #HAVING A HARD CODED KEY IS INSECURE - SHOULD BE ENV/EXTERNAL VARIABLE
    cypher_key=b'xqi9zRusHkcv30m050HwX82eMT0-LbeW4Y1qVVEzpw8='
    cypher=Fernet(cypher_key)
    encrypted_payload=cypher.encrypt(payload.encode('utf-8'))
    return(encrypted_payload.decode())
```

```
mqttc.publish(base_topic+"/temperature", encrypt_payload(temp_json))
```

Raspberry Pi: Creating MQTT Clients in Python: Publishing Client

- We'll use **Paho** MQTT python client from Eclipse
- Import the **client class** to provide functions to publish messages and subscribe to topics on MQTT brokers.
- Create instance and connect to a broker:

```
import paho.mqtt.client as mqtt #import the client1
broker_address="192.168.1.184"
#broker_address="iot.eclipse.org" #use external broker
client = mqtt.Client("P1") #create new instance
client.connect(broker_address) #connect to broker
client.publish("house/main-light","OFF")#publish
```

Raspberry Pi: Creating MQTT Clients in Python: Subscribing Client Callback

- When the client receives messages it generate the on_message callback.

```
import paho.mqtt.client as mqtt

def on_message(client, userdata, message):
    print("message received " ,str(message.payload.decode("utf-8")))
    print("message topic=",message.topic)
    print("message qos=",message.qos)

def main():
    client = mqtt.Client(client_id=CLIENT_ID)
    client.on_connect = on_connect
    client.on_message = on_message
    client.tls_set("./mosquitto.org.crt")
    client.connect(MQTT_BROKER, MQTT_PORT, keepalive=60)
    client.loop_forever()

if __name__ == "__main__":
    main()
```

Arduino: Creating MQTT Clients in a Sketch:

Publishing Client

```
#include <PubSubClient.h>
```

Uses PubSubClient from
Arduino Library

```
WiFiClient wifiClient;  
PubSubClient client(wifiClient);
```

Construct MQTT client using
wifiClient

```
client.setServer(mqttServer, mqttPort);
```

Set MQTT Server and port

```
client.connect(connectionId)
```

Create Connection

```
client.publish("/fxwalsh/temperature", message);
```

Publish message to topic!

Javascript/Node.js MQTT Client

```
const mqtt = require('mqtt');

const client = mqtt.connect('mqtt://broker.emqx.io');
const topic = '/fxwalsh/temperature';

client.on('connect', function () {
  client.subscribe(topic, function (err) {
    if (!err) {
      console.log(`Subscribed to ${topic}`);
    }
  });
});

client.on('message', function (topic, message) {
  // message is a buffer
  console.log(message.toString());
});
```